Strategy Pattern in JavaScript

What is the Strategy Pattern?

The **Strategy Pattern** is a behavioural design pattern that allows you to define a **family of algorithms**, put each algorithm into a separate class or function, and make them interchangeable at runtime.

In simple terms:

> "Create multiple ways of doing something and choose the behaviour you need without changing the main object."

Instead of having one class full of conditional logic:

```
if(type === "creditCard") {

    payWithCreditCard();

}

else if(type === "paypal") {

    payWithPaypal();

}

else if(type === "crypto") {

    payWithCrypto();

}
```

The Strategy Pattern separates each behaviour:

```
Payment
   |
   +--> CreditCardStrategy
   |
   +--> PayPalStrategy
   |
   +--> CryptoStrategy
```

The payment system can switch strategies without changing its own code.

Why Does the Strategy Pattern Exist?

A common problem is when a class has too many different behaviours controlled by `if` statements.

Example:

```
class ShippingCalculator {

    calculate(type, weight){

        if(type === "standard"){

            return weight * 2;

        }

        if(type === "express"){

            return weight * 5;

        }

        if(type === "overnight"){

            return weight * 10;

        }

    }

}
```

Problems:

- The class grows larger over time.
- Adding new behaviours requires modifying existing code.
- Testing becomes harder.
- Logic becomes tightly coupled.

With Strategy:

```
ShippingCalculator
```

```
        |

        v

Shipping Strategy

        |

 +-------------+
 |      |      |
Standard Express Overnight
```

Each algorithm is separate.

Basic Strategy Pattern Example

Payment System Example

Imagine an online store supporting:

- Credit card payments.
- PayPal payments.
- Cryptocurrency payments.

Without Strategy:

```
class Payment {


    pay(type, amount){


        if(type === "card"){

            console.log(
                `Paid ${amount} by card`
            );

        }


        if(type === "paypal"){

            console.log(
                `Paid ${amount} by PayPal`
            );

        }
```

```
    }

}
```

The class handles every payment type.

With Strategy:

```
payment.setStrategy(new PayPalStrategy());

payment.pay(100);
```

The payment method can change dynamically.

Strategy Classes

Each strategy contains one algorithm.

Credit Card Strategy

```
class CreditCardStrategy {


    pay(amount){

        console.log(
            `Paid €${amount} using Credit Card`
        );

    }

}
```

PayPal Strategy

```
class PayPalStrategy {


    pay(amount){
```

```
        console.log(
            `Paid €${amount} using PayPal`
        );

    }

}
```

Crypto Strategy

```
class CryptoStrategy {


    pay(amount){

        console.log(
            `Paid €${amount} using Crypto`
        );

    }

}
```

Context Class

The Context uses a strategy but does not know the implementation details.

```
class Payment {


    constructor(strategy){

        this.strategy = strategy;

    }



    setStrategy(strategy){

        this.strategy = strategy;

    }
```

```
    pay(amount){
        this.strategy.pay(amount);
    }
}
```

Using the Strategy Pattern

```
const payment =
    new Payment(
        new CreditCardStrategy()
    );

payment.pay(100);
```

Output:

```
Paid €100 using Credit Card
```

Change strategy:

```
payment.setStrategy(
    new PayPalStrategy()
);

payment.pay(100);
```

Output:

```
Paid €100 using PayPal
```

Change again:

```
payment.setStrategy(
    new CryptoStrategy()
);


payment.pay(100);
```

Output:

```
Paid €100 using Crypto
```

How This Works Step-by-Step

1. Create the Context

```
const payment = new Payment();
```

The Payment class is responsible for:

- Managing the payment process.
- Calling the chosen strategy.

It does not know how payment happens.

---

2. Choose a Strategy

```
payment.setStrategy(
    new PayPalStrategy()
);
```

The strategy is injected.

Now:

```
Payment
   |
   v
PayPalStrategy
```

---

3. Execute Behaviour

```
payment.pay(100);
```

Internally:

```
this.strategy.pay(amount);
```

The chosen strategy handles the operation.

---

4. Swap Behaviour

Change:

```
new PayPalStrategy()
```

to:

```
new CryptoStrategy()
```

The Payment class does not change.

Strategy Pattern Using JavaScript Functions

Because JavaScript treats functions as first-class objects, strategies do not always need classes.

Example:

```
const discountStrategies = {

    normal(price){
        return price;
    },

    student(price){
        return price * 0.8;
```

```
    },

    premium(price){
        return price * 0.5;
    }
};
```

Context:

```
class ShoppingCart {

    constructor(discountStrategy){
        this.discountStrategy =
            discountStrategy;
    }

    checkout(price){
        return this.discountStrategy(price);
    }
}
```

Usage:

```
const cart =
    new ShoppingCart(
        discountStrategies.student
    );

console.log(
    cart.checkout(100)
);
```

Output:

```
80
```

A More Realistic JavaScript Example

File Compression System

Imagine an application supporting:

- ZIP compression.
- RAR compression.
- TAR compression.

Without Strategy:

```
class Compressor {

    compress(type,file){

        if(type==="zip"){

        }

        if(type==="rar"){

        }

    }

}
```

With Strategy:

```
class ZipCompression {

    compress(file){

        console.log(
```

```
            `${file} compressed using ZIP`
        );

    }

}



class RarCompression {


    compress(file){

        console.log(
            `${file} compressed using RAR`
        );

    }

}
```

Context:

```
class FileCompressor {


    constructor(strategy){

        this.strategy = strategy;

    }


    compress(file){

        this.strategy.compress(file);

    }

}
```

Usage:

```
const compressor =
    new FileCompressor(
```

```
        new ZipCompression()
    );


compressor.compress("photo.jpg");
```

Output:

```
photo.jpg compressed using ZIP
```

Common Uses of Strategy Pattern in JavaScript

1. Payment Processing

Example:

```
payment.setStrategy(
    new StripePayment()
);
```

Different algorithms:

- Stripe.
- PayPal.
- Bank transfer.
- Crypto.

Problem solved:

> Add new payment methods without changing payment logic.

---

2. Sorting Algorithms

Example:

```
sorter.setStrategy(
    new QuickSort()
);
```

Different strategies:

- Bubble sort.
- Merge sort.

- Quick sort.

Problem solved:

> Change algorithms without changing the sorting system.

---

3. Authentication Methods

Example:

```
auth.setStrategy(
    new GoogleAuth()
);
```

Different strategies:

- Username/password.
- Google login.
- Microsoft login.
- GitHub login.

Problem solved:

> Support multiple authentication providers.

---

4. Discount Systems

Example:

```
checkout.setStrategy(
    new StudentDiscount()
);
```

Different strategies:

- Student discount.
- VIP discount.
- Seasonal discount.

Problem solved:

> Change pricing rules without rewriting checkout code.

---

5. Navigation / Routing

Example:

```
navigator.setStrategy(
    new WalkingRoute()
);
```

Different strategies:

- Walking.
- Driving.
- Cycling.

Problem solved:

Swap algorithms based on user preference.

Problems Strategy Pattern Solves

Problem 1: Too Many Conditional Statements

Without Strategy:

```
if(payment==="paypal")

if(payment==="card")

if(payment==="crypto")
```

The code becomes difficult to maintain.

With Strategy:

```
payment.pay();
```

The strategy decides how.

Problem 2: Changing Behaviour at Runtime

Example:

```
cart.setDiscount(
    new ChristmasDiscount()
);
```

Behaviour changes without changing the object.

Problem 3: Violating Open/Closed Principle

The Open/Closed Principle states:

> Software should be open for extension but closed for modification.

Without Strategy:

Adding a new payment type:

```
Modify Payment class
```

With Strategy:

Add:

```
class ApplePayStrategy {

}
```

Existing code stays unchanged.

Strategy vs Factory Pattern

Strategy	Factory
Changes behaviour	Creates objects
Focuses on algorithms	Focuses on instantiation
Objects already exist	Creates objects
Behaviour selection	Object selection

Strategy:

```
payment.setStrategy(
    new PayPalStrategy()
);
```

Question:

> "How should this operation happen?"

Factory:

```
PaymentFactory.create("paypal");
```

Question:

> "Which object should I create?"

Strategy vs State Pattern

Strategy	State
Behaviour selected externally	Behaviour changes internally
User chooses strategy	Object changes state
Usually independent algorithms	Represents different states

Strategy:

```
sorter.setStrategy(
    quickSort
);
```

State:

```
player.changeState(
    attacking
);
```

Strategy vs Template Method

Strategy	Template Method
Uses composition	Uses inheritance
Swap behaviours dynamically	Define algorithm structure
More flexible	More rigid

Strategy:

```
object.strategy.execute();
```

Template:

```
class BaseClass {

    algorithm(){

    }

}
```

Disadvantages of Strategy Pattern

1. More Classes

Instead of:

```
Payment
```

You now have:

```
Payment

CreditCardStrategy

PayPalStrategy

CryptoStrategy
```

---

2. Client Must Understand Strategies

The client needs to know which strategy to choose.

---

3. Can Be Overkill

For simple logic:

```
if(type==="small")
```

may be perfectly fine.

When Should You Use Strategy?

Use Strategy when:

✅ You have multiple algorithms for the same task
✅ Behaviour changes depending on conditions
✅ You want to avoid large if/else statements
✅ You need runtime behaviour switching

Examples:

- Payment methods.
- Authentication providers.
- Sorting algorithms.
- Discount calculations.
- Compression algorithms.

Avoid Strategy when:

❌ There is only one behaviour
❌ The logic is simple
❌ Creating multiple classes adds unnecessary complexity

Key Takeaway

The Strategy Pattern separates:

```
Object
   |
   v
Strategy Interface
```

```
    |
    +--> Algorithm A
    |
    +--> Algorithm B
    |
    +--> Algorithm C
```

Instead of:

```
One Huge Class

if()
else if()
else if()
```

In JavaScript, the Strategy Pattern is commonly implemented using:

1. Separate strategy classes.
2. Functions passed as strategies.
3. Dependency injection.

It is mainly used to **make algorithms interchangeable and remove complex conditional logic while keeping code flexible and maintainable**.